AI-Powered Smart Retail & Customer Intelligence Platform

13(A) - Puneet Gurbani

Submitted by

Aman Patre

Application No.: IN26013953

Registration No.: 23BCE1180

GitHub Link: https://github.com/AmanPatre/AI-Powered-Smart-Retail-Customer-Intelligence-Platform

Certificate

This is to certify that the project report entitled “AI-Powered Smart Retail & Customer Intelligence Platform” is a bonafide record of the work carried out by Aman Patre (Registration No.: 23BCE11840, Application No.: IN26013953) as part of the Major Project requirement. The work presented in this report is an original piece of work carried out under the guidance of the concerned faculty/mentor and has not been submitted elsewhere for the award of any other degree or diploma.

Date: 05/08/2026

Signature of Mentor / Guide

Puneet Gurbani

Declaration

I, Aman Patre, hereby declare that the project report titled “AI-Powered Smart Retail & Customer Intelligence Platform”, submitted in fulfilment of the requirements for the Major Project, is an authentic record of my own work carried out using Python and associated machine learning, computer vision, and web-service libraries. All sources of information, code libraries, datasets, and reference material used in this project have been duly acknowledged in the References section of this report.

Name: Aman Patre

Application No.: IN26013953

Registration No.: 23BCE11840

Acknowledgement

I would like to express my sincere gratitude to my mentors and faculty for their continuous guidance, support, and valuable feedback throughout the course of this project. Their insights into applied machine learning, system design, and deployment practices were instrumental in shaping the direction of this work.

I also extend my thanks to the open-source community whose libraries, frameworks, and documentation including OpenCV, TensorFlow, scikit-learn, FastAPI, and HuggingFace made the implementation of this platform possible within the project timeline. Finally, I am grateful to my peers for their constructive discussions during the design and testing phases of this project.

Aman Patre

Table of Contents

Abstract

Retail businesses today generate large volumes of unstructured data through in-store footfall, customer feedback, and support conversations, yet most small and mid-sized retailers lack an integrated system to convert this data into actionable intelligence. This project presents the design and implementation of an AI-Powered Smart Retail & Customer Intelligence Platform, a unified system that combines computer vision, natural language processing, and conversational AI behind a single production-style REST API. The platform recognizes returning customers through face recognition, classifies product images into categories using transfer learning, analyzes the sentiment of customer reviews and feedback, and answers frequently asked questions through a hybrid rule-based and machine-learning chatbot. All models are wrapped in a unified inference pipeline, serialized using Pickle/Joblib, and exposed through a FastAPI service that can be containerized with Docker for cloud deployment. The report documents the system architecture, module-wise implementation, technology stack, development timeline, ethical considerations around facial recognition, and the evaluation approach used to assess the platform, concluding with directions for future enhancement.

1. Introduction

The retail industry is undergoing rapid digital transformation, with businesses increasingly relying on artificial intelligence to understand customer behaviour, streamline operations, and personalize service. While large retailers such as Amazon and Walmart have invested heavily in AI-driven analytics, similar capabilities remain largely inaccessible to smaller retail and e-commerce businesses due to the cost and complexity of building fragmented, single-purpose AI tools.

This project addresses that gap by building a single, deployable AI platform that a retail or e-commerce business could realistically adopt. Rather than treating computer vision, natural language processing, and conversational AI as isolated academic exercises, the platform integrates all three into one cohesive product: a customer walks in and is recognized by the system, their feedback is automatically analyzed for sentiment, and their queries are answered by an intelligent chatbot — all served through a single, documented API.

The project was structured so that every core topic of the course syllabus (OpenCV fundamentals, image classification, face recognition, text preprocessing, sentiment analysis, chatbot design, ML pipelines, model serialization, and API deployment) maps directly onto a working module of the final system, ensuring that theoretical concepts were reinforced through an end-to-end, real-world implementation.

Beyond the technical build, the project was also treated as an exercise in software engineering discipline: version-controlled code, a documented API contract, automated tests, containerized deployment, and a written evaluation of ethical trade-offs. This mirrors the expectations of an industry-grade AI product rather than a collection of standalone notebooks, and reflects the growing demand for AI practitioners who can take a model from a Jupyter notebook to a deployed, monitorable service.

The remainder of this report is organized as follows: Section 2 reviews related work and existing approaches in retail AI; Section 3 states the problem being solved; Section 4 lists the project objectives; Section 5 details functional and non-functional requirements; Section 6 describes the system architecture; Section 7 covers the technology stack and the rationale behind each choice; Section 8 provides a module-by-module account of the implementation; Section 9 describes the datasets used; Section 10 shows the project folder structure; Section 11 presents the development timeline; Section 12

describes the testing strategy; Section 13 discusses results and evaluation; Section 14 addresses ethical considerations; Section 15 discusses limitations and challenges faced; and Section 16 concludes with a summary and directions for future work.

2. Literature Survey / Related Work

A review of existing approaches informed the design of this platform. Prior work in retail analytics can broadly be grouped into three categories relevant to this project: computer-vision-based customer analytics, NLP-based customer feedback analysis, and conversational agents for customer support.

2.1 Computer Vision in Retail

Industry systems such as Intel's OpenVINO-based retail analytics toolkits demonstrate how face detection, person counting, and shelf-inventory monitoring can be combined using pre-trained deep learning models to provide store-traffic and engagement analytics. These systems typically rely on lightweight, pre-trained detection models (e.g., face-detection-adas, person-detection-retail) that are optimized for edge deployment, which motivated the choice of a similarly lightweight architecture (MobileNetV2, Haar Cascades, LBPH/dlib-based recognition) for this project rather than heavier research-grade models that would be difficult to serve in real time.

2.2 Sentiment Analysis for Customer Feedback

Sentiment classification of product reviews is a well-studied NLP task. Classical approaches using TF-IDF features with Logistic Regression or Support Vector Machines remain competitive baselines for short-text sentiment classification, while transformer-based models such as BERT and its distilled variants (DistilBERT) offer higher accuracy at the cost of additional compute and latency. This project adopts a two-tier strategy: a fast, interpretable TF-IDF baseline for the core deliverable, with DistilBERT identified as an optional upgrade path once the baseline pipeline is validated end-to-end.

2.3 Retail Chatbots

Most production customer-support chatbots use a hybrid design rather than a purely generative or purely rule-based approach: a rule/intent-matching layer handles high-confidence, high-frequency queries (such as store hours or return policy) with deterministic, auditable responses, while a machine-learning classifier acts as a fallback for queries that do not match a known pattern. This hybrid pattern was adopted for the chatbot module in this project because it offers predictable behaviour for common queries while still generalizing to varied customer phrasing.

2.4 Gap Identified

While each of the above techniques is individually well documented, most academic projects and even several commercial tools treat them as separate products. The gap this project addresses is architectural integration: unifying face recognition, product classification, sentiment analysis, and a chatbot behind one API and one deployment pipeline, so that a retailer can adopt a single system rather than stitching together multiple vendors or tools.

3. Problem Statement

Retailers need a way to (a) identify and track returning customers without manual intervention, (b) automatically classify product images for catalogue and inventory purposes, (c) understand customer

sentiment from reviews and chat logs at scale, and (d) provide instant, automated responses to common customer queries. Existing solutions typically address only one of these needs in isolation and are rarely packaged in a way that is easy to deploy or extend.

Small and mid-sized retailers, in particular, face a practical barrier: even where individual open-source models exist for each of these tasks, integrating them into a single, reliable, and deployable service requires software engineering effort that is often out of reach without a dedicated engineering team. There is, therefore, a need for a reference implementation that shows how these capabilities can be combined into one coherent, well-documented, and deployable system using freely available tools.

The objective of this project is to design, implement, and deploy a single AI-driven platform that solves all four problems together, using a modular architecture so that each capability (vision, language, and conversation) can be developed, tested, and improved independently while still operating as part of one unified, production-style system.

4. Objectives

- Build a customer recognition module using face detection and recognition to log store visits and identify returning customers.
- Develop an image classification model to automatically categorize product images using transfer learning.
- Design an NLP pipeline for cleaning and analyzing customer reviews/feedback to determine sentiment (positive, negative, neutral).
- Build a hybrid rule-based and ML-driven chatbot capable of answering frequently asked customer queries.
- Integrate all modules into a single, unified inference pipeline with consistent model serialization.
- Expose the platform through a documented, production-style REST API using FastAPI.
- Containerize and deploy the platform using Docker, with consideration for authentication and basic security.
- Evaluate the ethical implications of facial recognition in a retail context, including consent, privacy, and bias.
- Document the system thoroughly, including architecture diagrams, API specifications, and a written evaluation of design trade-offs.

5. Requirement Analysis

5.1 Functional Requirements

- The system shall detect a face from an uploaded image or video frame and return a match against a stored customer database, or register it as a new visitor.
- The system shall classify an uploaded product image into one of a pre-defined set of categories with an associated confidence score.
- The system shall accept free-text customer feedback and return a sentiment label (Positive / Negative / Neutral) with a confidence score.
- The system shall accept a free-text customer message and return an appropriate chatbot response, using rule-based matching where possible and a trained classifier as fallback.
- The system shall expose all the above capabilities via documented REST API endpoints that accept and return JSON (and image uploads where applicable).

- The system shall provide an aggregate statistics endpoint summarizing visit counts and sentiment distribution over a given period.

5.2 Non-Functional Requirements

- Performance: individual inference requests (face match, classification, sentiment) should return within a few seconds on standard hardware to remain usable in a live retail setting.
- Scalability: the API layer should be stateless where possible so that additional instances can be deployed behind a load balancer if traffic increases.
- Maintainability: each AI capability should be encapsulated in its own service module so that a model can be retrained or replaced without affecting the other modules.
- Security: endpoints should require an API key or authentication header, and uploaded images/text should be validated before being passed to a model.
- Portability: the system should run consistently across environments through containerization with Docker.
- Privacy: biometric data (face encodings) should be stored separately from other application data and only used for the stated recognition purpose.

5.3 Hardware Requirements

Component	Minimum Specification
Processor	Quad-core CPU (Intel i5 / equivalent or higher)
RAM	8 GB minimum (16 GB recommended for model training)
GPU (optional)	CUDA-capable GPU recommended for faster CNN/transformer training
Storage	10 GB free space for datasets, models, and dependencies
Camera	Webcam (for live face-recognition testing)

5.4 Software Requirements

Component	Specification
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Language Runtime	Python 3.9 or higher
Package Manager	pip / conda
Containerization	Docker Engine 20.x or higher
API Testing	Postman / cURL / Swagger UI (auto-generated by FastAPI)
Version Control	Git and GitHub

6. System Architecture

The platform follows a layered, service-oriented architecture. A client layer (dashboard, API testing tools such as Postman, or a live webcam feed) sends REST requests to a central FastAPI gateway. The gateway exposes dedicated endpoints for face recognition, product classification, sentiment analysis,

chatbot interaction, and aggregate dashboard statistics. Each endpoint delegates to one of three underlying modules — the Computer Vision (CV) module, the Natural Language Processing (NLP) module, or the Chatbot module — all of which read from and write to a shared storage layer containing customer visit logs, review/chat data, and serialized model files.

Figure 1: High-Level System Architecture

Layer	Components	Responsibility
Client Layer	Dashboard / Postman / Webcam feed	Initiates REST requests to the API gateway
API Gateway	FastAPI: /recognize-face, /classify-product, /analyze-sentiment, /chatbot, /dashboard/stats	Routes requests, validates input/output via Pydantic schemas
CV Module	OpenCV capture, face embeddings, CNN classifier	Face detection/recognition, product image classification
NLP Module	Text cleaning, TF-IDF/BERT, sentiment model	Review/feedback preprocessing and sentiment classification
Chatbot Module	Intent matching, rule + ML hybrid, FAQ retrieval	Answers customer queries via rule-based and ML fallback logic
Storage Layer	customer_visits, reviews, chat_logs, model files (.pkl / .h5)	Persists logs, datasets, and serialized trained models

Each module is independently testable and communicates with the API gateway through well-defined function calls, which keeps the vision, language, and conversational components decoupled while still allowing them to operate under one unified pipeline at inference time.

6.1 Typical Request Flow

A typical end-to-end interaction proceeds as follows: (1) a client sends an image or text payload to the relevant FastAPI endpoint; (2) FastAPI validates the payload against its Pydantic schema and rejects malformed requests before any model is invoked; (3) the request is routed to the corresponding service function (cv_service, nlp_service, or chatbot_service); (4) the service function retrieves the already-loaded model instance from the unified pipeline and performs inference; (5) the result is formatted into the response schema and returned as JSON; and (6) where relevant (e.g., a face match or a new chat interaction), the event is logged to the storage layer for later aggregation in the /dashboard/stats endpoint.

This request flow keeps latency low by ensuring models are loaded exactly once at startup rather than on every request, and it keeps the system auditable, since every recognized visit, classified product, and chatbot interaction is logged with a timestamp.

7. Technology Stack

The technology choices for this project were guided by three criteria: (1) suitability for rapid prototyping within an academic timeline, (2) availability of strong community support and documentation, and (3) feasibility of deployment on free or low-cost infrastructure. Table 3 summarizes the stack, and the subsections that follow justify the key choices.

Category	Tools / Libraries
Programming Language	Python 3
Computer Vision	OpenCV, Haar Cascades, face_recognition (dlib-based) / LBPH
Deep Learning	TensorFlow / Keras (or PyTorch), MobileNetV2 (transfer learning)
NLP	spaCy / NLTK, TF-IDF, scikit-learn (Logistic Regression / SVM), HuggingFace DistilBERT (optional upgrade)
Chatbot	Rule-based intent matching, TF-IDF + classifier for ML fallback
Model Serialization	Pickle, Joblib, native .h5 / .pt formats
API Framework	FastAPI with Pydantic schemas, auto-generated Swagger docs
Deployment	Docker, Render / Railway / AWS EC2 / Google Cloud Run
CI/CD (optional)	GitHub Actions for lint, test, and image build on push

7.1 Why Python

Python was selected as the sole implementation language because it provides mature, well-tested libraries for every capability required by the platform — OpenCV and dlib for vision, scikit-learn/TensorFlow for machine learning, and FastAPI for serving — removing the need to bridge multiple languages across the pipeline.

7.2 Why FastAPI over Flask

FastAPI was preferred over Flask for three concrete reasons: native support for asynchronous request handling (useful when a request triggers a model inference call), automatic request and response validation through Pydantic models (reducing manual input-checking code), and automatically generated interactive API documentation (Swagger UI at /docs and ReDoc at /redoc), which is valuable both for testing during development and for demonstrating the API during evaluation.

7.3 Why MobileNetV2 for Image Classification

MobileNetV2 was chosen over larger architectures such as ResNet50 or VGG16 because it is specifically designed for constrained, low-latency inference while still benefiting from transfer learning on ImageNet-pretrained weights. This keeps both training time and inference latency low enough for a near-real-time retail use case.

7.4 Why a Hybrid TF-IDF and Transformer Approach for NLP

A TF-IDF plus classical classifier baseline was implemented first because it trains in seconds, is fully interpretable, and requires no GPU — making it well suited to the constrained project timeline. DistilBERT was retained as an optional upgrade path for teams or future iterations that need higher sentiment-classification accuracy and have access to GPU resources.

8. Module-wise Implementation

8.1 Module A — Computer Vision

A1. OpenCV Basics

The foundation of the vision pipeline is a reusable set of OpenCV utilities (cv_utils.py) that capture webcam or video input and apply preprocessing operations such as grayscale conversion, resizing, Gaussian blur, and Canny edge detection. Haar Cascade classifiers are used for initial face bounding-box detection before frames are passed downstream to the recognition and classification models. A representative snippet of the preprocessing utility is shown below.

```
def preprocess_frame(frame):     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)     blurred = cv2.GaussianBlur(gray, (5, 5), 0) edges = cv2.Canny(blurred, 50, 150)         faces = face_cascade.detectMultiScale(gray, 1.1, 5)     return gray, edges, faces
```

A2. Image Classification

Product images are classified into categories (e.g., shoes, bags, electronics, clothing, groceries) using transfer learning on MobileNetV2, chosen for its balance of speed and accuracy in deployment-constrained environments. The convolutional base is loaded with ImageNet-pretrained weights and frozen, while a new classification head (global average pooling, a dense layer, and a softmax output layer) is trained on the product-category dataset. The model outputs a predicted category along with a confidence score and is serialized as product_classifier.h5 for use in the inference pipeline.

```
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224,224,3)) base_model.trainable = False x = GlobalAveragePooling2D()(base_model.output) x = Dense(128, activation='relu')(x) output = Dense(num_classes, activation='softmax')(x) model = Model(inputs=base_model.input, outputs=output) model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

A3. Face Recognition

Customer recognition is implemented using the face_recognition library (built on dlib) or an LBPH recognizer as a lighter-weight alternative. The pipeline detects a face, generates a 128-dimensional numerical encoding, and compares it against a database of stored customer encodings (face_db.pkl) using Euclidean distance, matching against a configurable similarity threshold. Each successful match is logged with a timestamp for loyalty analytics, and unmatched faces are optionally registered as new visitors. A demo dataset of consenting sample faces (such as an LFW subset) was used to validate this module.

```
unknown_encoding = face_recognition.face_encodings(image)[0] distances = face_recognition.face_distance(known_encodings, unknown_encoding) best_match = np.argmin(distances) if distances[best_match] < THRESHOLD:     return known_ids[best_match]   # returning customer else:     return register_new_customer(unknown_encoding)
```

8.2 Module B — Natural Language Processing

B1. Text Preprocessing

Customer reviews and chat text are cleaned through a standard NLP pipeline: lowercasing, punctuation removal, stopword filtering, lemmatization (using spaCy/NLTK), and tokenization, applied to a reviews dataset such as a Women's E-Commerce Clothing Reviews-style corpus. This preprocessing step is critical since raw review text is noisy and inconsistent, containing typos, emojis, and inconsistent casing that would otherwise degrade downstream model performance.

B2. Sentiment Analysis

A baseline sentiment classifier is trained using TF-IDF features with Logistic Regression or SVM, with an optional upgrade path to a fine-tuned DistilBERT transformer for higher accuracy. The TF-IDF vectorizer converts cleaned review text into a sparse numerical representation weighted by term frequency and inverse document frequency, which is then fed into the classifier. The model outputs a

Positive, Negative, or Neutral label along with a confidence score, and is serialized as sentiment_model.pkl together with its vectorizer.

```
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2)) X = vectorizer.fit_transform(cleaned_reviews)
clf = LogisticRegression(max_iter=1000) clf.fit(X, labels) joblib.dump(clf, 'sentiment_model.pkl')
joblib.dump(vectorizer, 'vectorizer.pkl')
```

B3. Chatbot

The chatbot uses a hybrid design: a rule-based layer handles common FAQs (order status, return policy, store hours) through direct intent matching, while a machine-learning fallback — a TF-IDF plus classifier model trained on a custom intents.json file — handles queries that do not match a predefined rule, improving robustness to varied customer phrasing. The intents file defines each intent as a set of example patterns and a corresponding response template, allowing new FAQ categories to be added without retraining the entire system.

```
{ "intents": [ {"tag": "store_hours", "patterns": ["What time do you open", "store hours", "when do you close"],
"responses": ["We are open 9 AM to 9 PM, Monday to Saturday."]}, {"tag": "return_policy", "patterns": ["How
do I return an item", "refund policy"], "responses": ["Items can be returned within 30 days with a valid receipt."]}
] }
```

8.3 Module C — ML Pipeline and Deployment

C1. Unified Pipeline

All three modules (face recognition, product classification, and sentiment/chatbot models) are wrapped in a single pipeline.py that loads every model once at application startup, avoiding repeated load overhead on each request. This design follows the singleton pattern for model objects, ensuring that memory-heavy models such as the CNN classifier are loaded exactly once per running instance.

C2. Model Serialization

scikit-learn models are serialized with Joblib, deep learning models are saved in their native .h5/.pt formats, and lightweight artifacts such as face encodings and chatbot intents are stored using Pickle. This consistent, per-model-type serialization strategy makes it straightforward to swap or retrain any single model without altering the loading logic of the others.

C3. API Layer

FastAPI was chosen over Flask for its native async support, automatic request/response validation via Pydantic, and auto-generated Swagger documentation. The platform exposes the following endpoints:

- POST /recognize-face — accepts an uploaded image and returns the matched customer ID or a new-visitor status.
- POST /classify-product — accepts an uploaded image and returns the predicted product category.
- POST /analyze-sentiment — accepts text input and returns the predicted sentiment label and confidence.
- POST /chatbot — accepts a customer message and returns the chatbot's reply.
- GET /dashboard/stats — returns aggregate visit and sentiment statistics in JSON for a simple analytics dashboard.

Each endpoint is backed by a Pydantic request/response schema, which both validates incoming data (e.g., rejecting malformed image uploads or empty text fields) and produces the OpenAPI schema used to auto-generate the Swagger documentation at /docs.

C4. Deployment

The application is containerized using Docker (Dockerfile and requirements.txt) for portability, with deployment targets such as Render, Railway, AWS EC2, or Google Cloud Run selected for their student-friendly free tiers. A basic API key or authentication header is added to the endpoints to simulate production-grade security, and GitHub Actions can optionally be configured to lint, test, and build the Docker image on every push.

```
FROM python:3.10-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt
COPY . . EXPOSE 8000 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

9. Dataset Description

Module	Dataset	Description
Image Classification	5-class product image set (Fashion-MNIST style / scraped set)	Shoes, bags, electronics, clothing, groceries — used to train the MobileNetV2 classifier
Face Recognition	LFW subset (demo, consenting sample faces)	Used to validate encoding generation and matching logic without using real customer data
Sentiment Analysis	Women's E-Commerce Clothing Reviews-style corpus	Labelled product reviews used to train and evaluate the TF-IDF + classifier sentiment model
Chatbot	Custom intents.json	Hand-authored set of FAQ intents, example patterns, and response templates for the retail domain

Each dataset was split into training and validation subsets (typically an 80/20 split) to allow model performance to be measured on unseen data before being serialized for use in the inference pipeline.

10. Project / Folder Structure

```
smart-retail-ai/ ├── app/ │ ├── main.py # FastAPI entrypoint │ ├── routers/ # vision.py,
nlp.py, chatbot.py │ ├── models/ # product_classifier.h5, face_db.pkl, │ │ #
sentiment_model.pkl, chatbot_model.pkl │ ├── services/ # cv_service.py, nlp_service.py,
chatbot_service.py │ └── schemas.py ├── notebooks/ # training / experiment notebooks ├── data/
# reviews.csv, intents.json ├── tests/ # test_endpoints.py ├── requirements.txt ├── README.md
└── .github/workflows/deploy.yml
```

11. Development Timeline

Days	Task
1 – 2	Data collection, exploratory data analysis, and preprocessing for both CV and NLP datasets
3 – 4	Train and evaluate the image classifier; set up the face recognition pipeline
5 – 6	Train the sentiment model; build chatbot intents and model
6	Build the unified pipeline; serialize all models

Days	Task
7	Build FastAPI endpoints, API documentation, and input validation
8	Write automated tests; build a minimal frontend/dashboard (Streamlit or React)
9	Final report, architecture diagram, demo video, and README

12. Testing Strategy

Testing was carried out at two levels: unit-level testing of individual model functions and integration-level testing of the API endpoints, using pytest and FastAPI's built-in TestClient. The goal was to verify both correctness (does the endpoint return the expected shape and type of response) and robustness (does the endpoint fail gracefully on invalid input).

12.1 Sample Test Cases

Test ID	Module / Endpoint	Input	Expected Result
TC-01	/classify-product	Valid product image	HTTP 200 with predicted category and confidence score
TC-02	/classify-product	Non-image file	HTTP 422 validation error
TC-03	/recognize-face	Image of an enrolled customer	HTTP 200 with matching customer ID
TC-04	/recognize-face	Image of an unenrolled face	HTTP 200 with new-visitor status
TC-05	/analyze-sentiment	"The product quality was excellent"	HTTP 200, sentiment = Positive
TC-06	/analyze-sentiment	Empty text field	HTTP 422 validation error
TC-07	/chatbot	"What are your store hours?"	HTTP 200 with the store-hours response
TC-08	/chatbot	Out-of-scope query	HTTP 200 with the ML-fallback or default response
TC-09	/dashboard/stats	GET request, no parameters	HTTP 200 with aggregate visit/sentiment JSON
TC-10	Any endpoint	Missing/invalid API key	HTTP 401 Unauthorized

These test cases were automated using pytest fixtures so that the full suite could be re-run after any model retraining or code change, reducing the risk of regressions being introduced during iterative development.

13. Results and Discussion

Because the underlying models were trained on small, project-scale datasets rather than large production datasets, the reported results should be interpreted as a proof-of-concept validation of the architecture

and pipeline rather than a benchmark of production-grade accuracy. The table below summarizes representative evaluation results for each module.

Module	Metric	Observation
Product Image Classifier	Validation accuracy	High accuracy on the held-out validation split, benefiting from MobileNetV2 transfer learning
Face Recognition	Match accuracy on demo set	Reliable matching on clear, front-facing images; accuracy degrades with poor lighting or extreme angles
Sentiment Classifier	Validation accuracy / F1-score	Strong performance on clearly positive/negative reviews; more ambiguity on neutral/mixed reviews
Chatbot	Intent-match rate	High accuracy for in-scope FAQ intents; ML fallback handles paraphrased queries reasonably well
API Layer	Average response latency	Sub-second for text-based endpoints; image-based endpoints slightly higher due to preprocessing

These results confirm that the overall architecture — a shared pipeline serving multiple lightweight models through one API — is a viable pattern for a small-to-mid-scale retail deployment, while also highlighting the specific areas (lighting-sensitive face matching, neutral-sentiment ambiguity) that would need further data and tuning before a production rollout.

14. Ethical Considerations

Because the platform uses facial recognition to identify returning customers, the project explicitly considers the ethical implications of this capability. Key concerns include obtaining informed consent before enrolling a customer's face into the recognition database, ensuring collected biometric data is stored securely and only for its stated purpose, and being transparent with customers about what data is captured and how it is used.

The project also acknowledges the risk of bias in face recognition systems, which have been shown in prior research to perform less accurately across certain demographic groups, and recommends evaluating recognition accuracy across diverse subsets of any deployment dataset. For development and demonstration purposes, the system uses a demo dataset of consenting sample faces rather than real customer data, and any production deployment would need to implement clear consent workflows, data retention limits, and an opt-out mechanism for customers who do not wish to be recognized.

In addition to the face recognition module, the sentiment-analysis and chatbot modules process customer-generated text, which may occasionally include personally identifiable information. The system design therefore separates biometric data, review/chat text, and aggregate analytics into distinct storage concerns, so that access controls and retention policies can be applied independently to each category of data.

15. Evaluation Approach

The platform is evaluated against the criteria typically used to assess an applied AI system of this kind:

Criteria	Weight	Description
Model accuracy	25%	Per-module accuracy of the face recognizer, image classifier, and sentiment model
Code quality & pipeline design	20%	Modularity, readability, and maintainability of the unified pipeline
API design & documentation	15%	Completeness and clarity of the FastAPI endpoints and Swagger docs
Deployment	20%	Whether a working, containerized/live demo was achieved
Report: architecture, ethics, tradeoffs	10%	Depth of documentation on design decisions and ethical review
Presentation/demo	10%	Clarity and completeness of the final walkthrough

16. Limitations and Challenges

- Dataset scale: all models were trained on small, project-scale datasets rather than large production datasets, which limits generalization and means reported accuracy figures should be treated as indicative rather than production-grade.
- Face recognition sensitivity: recognition accuracy is sensitive to lighting conditions, camera angle, and image resolution, requiring a controlled capture environment for reliable results.
- Neutral sentiment ambiguity: the sentiment classifier performs less reliably on reviews with mixed or subtle sentiment compared to clearly positive or negative reviews.
- Chatbot coverage: the rule-based layer only covers intents explicitly defined in intents.json; queries far outside this scope depend entirely on the ML fallback, which has a lower ceiling on accuracy than a rule match.
- Compute constraints: training larger models (e.g., a fine-tuned DistilBERT) was constrained by the absence of dedicated GPU resources within the project timeline, motivating the lighter-weight baseline choices documented in Section 7.
- Time constraints: the nine-day sprint format required prioritizing a working end-to-end pipeline over exhaustive hyperparameter tuning of any single model.

17. Conclusion and Future Scope

This project demonstrates that a single, well-architected system can bring together computer vision, natural language processing, and conversational AI to solve multiple real retail problems at once — customer recognition, product cataloguing, sentiment monitoring, and customer support — without treating each as an isolated academic exercise. By wrapping all models behind a unified pipeline and a documented FastAPI service, the platform mirrors how such a system would realistically be built and deployed in industry.

The project also reinforced practical software engineering skills that extend beyond model training alone: designing a REST API contract, writing automated tests, containerizing an application, and documenting ethical trade-offs — all of which are essential to taking an AI prototype into a form that could realistically be evaluated or piloted by a business stakeholder.

17.1 Future Enhancements

- Upgrade the sentiment model to a fine-tuned DistilBERT transformer for higher accuracy on ambiguous/neutral reviews.
- Expand the product classifier to a larger and more diverse product catalogue with additional categories.
- Add a real-time analytics dashboard (e.g., using Streamlit or React) with richer visualizations of footfall, sentiment trends, and chatbot usage.
- Strengthen deployment security with OAuth2-based authentication, request rate limiting, and full CI/CD via GitHub Actions.
- Conduct a formal bias audit of the face recognition module across diverse demographic subsets before any real-world deployment.
- Introduce a feedback loop where chatbot fallback responses flagged as unhelpful are used to periodically retrain the intent classifier.

18. References

1. FastAPI Documentation — https://fastapi.tiangolo.com/

2. OpenCV Documentation — https://docs.opencv.org/

3. face_recognition library (dlib-based) — https://github.com/ageitgey/face_recognition

4. TensorFlow / Keras Applications: MobileNetV2 — https://www.tensorflow.org/api_docs/python/tf/keras/applications/mobilenet_v2

5. HuggingFace Transformers: DistilBERT — https://huggingface.co/docs/transformers/model_doc/distilbert

6. scikit-learn Documentation — https://scikit-learn.org/

7. Docker Documentation — https://docs.docker.com/

8. Intel IoT DevKit, "Smart Retail Analytics" (OpenVINO reference implementation) — https://github.com/intel-iot-devkit/smart-retail-analytics

9. spaCy Documentation — https://spacy.io/

10. NLTK Documentation — https://www.nltk.org/

11. Pydantic Documentation — https://docs.pydantic.dev/

12. Render / Railway / AWS EC2 / Google Cloud Run deployment documentation (respective official sites).

19. Appendix

19.1 Sample API Request/Response — /analyze-sentiment

```
Request: POST /analyze-sentiment {  "text": "The delivery was fast and the product quality is great!" }  Response: { "sentiment": "Positive",  "confidence": 0.94 }
```

19.2 Sample API Request/Response — /chatbot

Request: POST /chatbot { "message": "What is your return policy?" } Response: { "reply": "Items can be returned within 30 days with a valid receipt.", "matched_intent": "return_policy" }

19.3 Sample requirements.txt

fastapi==0.110.0 uvicorn==0.29.0 opencv-python==4.9.0.80 face-recognition==1.3.0 tensorflow==2.15.0 scikit-learn==1.4.1 spacy==3.7.4 nltk==3.8.1 joblib==1.3.2 pydantic==2.6.4 python-multipart==0.0.9

19.4 List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CV	Computer Vision
FAQ	Frequently Asked Question
JSON	JavaScript Object Notation
LBPH	Local Binary Patterns Histograms
ML	Machine Learning
NLP	Natural Language Processing
REST	Representational State Transfer
TF-IDF	Term Frequency – Inverse Document Frequency

19.5 Sample API Response — /dashboard/stats

{ "total_visits": 128, "returning_customers": 47, "sentiment_breakdown": { "positive": 68, "neutral": 21, "negative": 11 } }